

INF1022 - Analisadores Léxicos e Sintáticos

Análise Sintática

Jefferson Santos Edward Hermann Haeusler

PUC-Rio

Departamento de Informática

Nome dado ao processo de determinação se um texto ou uma sequência de caracteres pertence a uma certa linguagem ou não, indicando as regras que validam os mesmos de acordo com uma gramática formal da linguagem.

Análise Sintática

*Aparentemente é uma atividade **inata** no ser humano que parece contar com uma espécie de **gramática universal** como um mecanismo que dá suporte a esta competência cognitiva e linguística*

Teoria defendida por **Noam Chomsky** para explicar a competência linguística e a aquisição da linguagem pelo ser humano, em bases minimalistas. Chomsky cria em 1956 as **gramáticas gerativas** e com elas provê suporte matemático/formal e finalmente computacional a linguística.

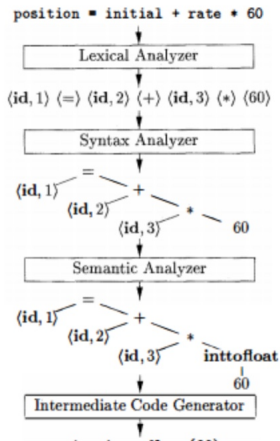
Chomsky, Noam (1956). "Three models for the description of language", IRE Transactions on Information Theory. 2 (3): 113–124.

Análise Sintática

Apesar do termo gramática ser bem antigo, pois fazia parte do Trivium e que antecedia o Quadrivium na antiguidade clássica e depois idade média, a definição que se tem hoje é devida a Chomsky (1956). É utilizada tanto em linguagens naturais quanto em linguagens artificiais, notadamente em Linguagens de Programação, de consulta, especificação formal, etc.

Elemento básico de um compilador para LPs é responsável por estruturar, de acordo com as regras de uma gramática, o programa lido como uma sequência de símbolos em uma hierarquia de comandos, declarações, expressões e unidades de programação. O código objeto é gerado somente após esta estruturação.

Estrutura de um Compilador (Foco na Sintaxe)

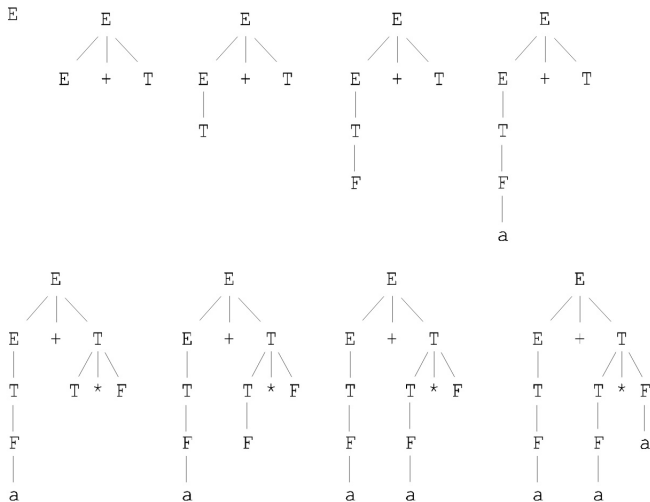


Técnicas de Análise Sintática (Parsing)

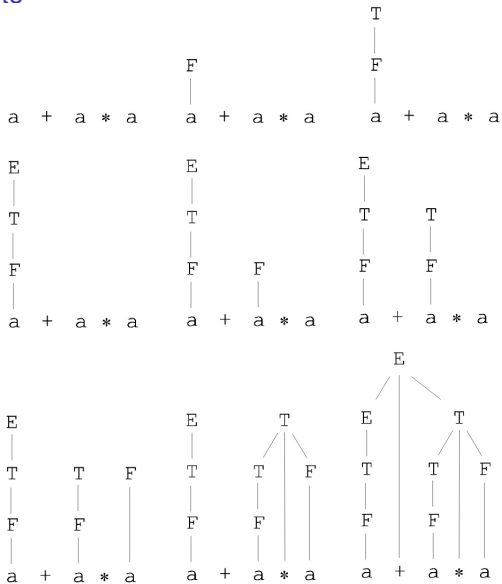
Basicamente duas:

- **Descendente** : Constrói a árvore de análise sintática (parsing tree) **da raiz para as folhas**. Corresponde a encontrar uma **derivação esquerda (leftmost)** para a cadeia de símbolos que está em análise, e;
- **Ascendente** : Constrói a árvore de análise sintática (parsing tree) **das folhas para a raiz**. Corresponde a encontrar uma **derivação direita (rightmost) invertida** para a cadeia de símbolos que está em análise.

Análise Descendente



Análise Ascendente



Revisitando a teoria

Teorema: Se $\mathcal{G}$ é **GLC** então existe um **APND** $\mathcal{A}_{\mathcal{L}}$ que aceita $L(\mathcal{G})$.

A gramática :
$$\begin{array}{lcl} E & \rightarrow & E + T \\ T & \rightarrow & T * F \\ F & \rightarrow & (E) \end{array} \quad \Bigg| \quad \begin{array}{l} T \\ F \\ a \end{array}$$
 gera $a + a * a$ e $\mathcal{A}_{\mathcal{G}}$ o aceita, via análise descendente construindo uma derivação mais a esquerda.

pilha	(resto da) entrada	derivação esquerda
E	a+a*a	E
E+T	a+a*a	$\Rightarrow$ E+T
T+T	a+a*a	$\Rightarrow$ T+T
F+T	a+a*a	$\Rightarrow$ F+T
a+T	a+a*a	$\Rightarrow$ a+T
+T	+a*a	
T	a*a	
T*F	a*a	$\Rightarrow$ a+T*F
F*F	a*a	$\Rightarrow$ a+F*F
a*F	a*a	$\Rightarrow$ a+a*F
*F	*a	
F	a	
a	a	$\Rightarrow$ a+a*a
ϵ	ϵ	

Conceitos básicos em Parser Descendente

Uso de configurações (*Pilha, resto – da – entrada*) para representar o processo de parsing. Exemplos: $(E, a + a * a)$, $(E + T, a + a * a)$, $(+T, +a * a)$, etc.

Duas formas de transição de configurações:

- **Expansão** (de não-terminal) pela regra $A \rightarrow \beta$ indica a transição de $(A\alpha, y)$ para $(\beta\alpha, y)$.
- **Verificação** (de terminal) a indica a transição de $(a\alpha, ay)$ para (α, y) .

O processo começa com $(E$, x)$ e termina com aceitação na configuração (ϵ, ϵ) , onde E é o símbolo inicial da gramática e x o string a ser analisado. Usa-se um símbolo $\$$ para marcar o fim do string.$

Informações gramaticais

Dada uma gramática livre de contexto $\mathcal{G}$, procura-se saber:

- Quais os não-terminais de $\mathcal{G}$ que geram (ou não) ϵ ;
- Para cada não-terminal A , quais são os símbolos terminais iniciadores de cadeias geradas a partir de A , $First(A)$;
- Que símbolos terminais são seguidores, de A , ou seja ocorrem após uma derivação de A , $Follow(A)$.

Iniciadores de um não-terminal: $First(A)$

$$First(A) = \{a : a \in \Sigma_G \text{ e } A \Rightarrow^* a\alpha\}$$

Cálculo de $First(A)$:

- Se $A \rightarrow a\alpha \in R_G$ então $a \in First(A)$;
- Se $A \rightarrow B_1 \dots B_n a\alpha \in R_G$, e todo $B_i \Rightarrow^* \epsilon$, $i = 1, \dots, n$, então $a \in First(A)$;
- Se $A \rightarrow B\alpha \in R_G$ e $a \in First(B)$ então $a \in First(A)$;
- Se $A \rightarrow B_1 \dots B_n B\alpha \in R_G$, e todo $B_i \Rightarrow^* \epsilon$, $i = 1, \dots, n$, e $a \in First(B)$ então $a \in First(A)$;

Seguidores de um não-terminal: $Follow(A)$

$$Follow(A) = \{a : a \in N_G \text{ e } S\$ \Rightarrow^* \alpha A a \beta, \alpha, \beta \text{ quaisquer}\}$$

Cálculo de $Follow$, supondo $B_1 \dots B_m = \gamma$ com $\gamma \Rightarrow^* \epsilon$ e $S\$ \Rightarrow^* \delta A \phi$:

- $\$ \in Follow(S)$, pois $S\$ \Rightarrow^* S\$$;
- Se $A \rightarrow \alpha B \gamma a \beta \in R_G$ então $a \in Follow(B)$
- Se $A \rightarrow \alpha B \gamma C \beta \in R_G$ então se $a \in First(C)$ então $a \in Follow(B)$;
- Se $A \rightarrow \alpha B \gamma \in R_G$ então se $a \in Follow(A)$ então $a \in Follow(B)$.

Análise Descendente LL(1)

Constrói-se uma tabela de regras na forma $M[A; a]$, para cada não terminal $A \in N_G$ e $a \in \Sigma_G$, de acordo com:

- Para cada regra $i : A \rightarrow \alpha$, tem-se $M[A; a] = i$, para cada $a \in \text{First}(\alpha)$;
- Para cada regra $i : A \rightarrow \alpha$, se $\alpha \Rightarrow^* \epsilon$, tem-se $M[A; a] = i$, para cada $a \in \text{Follow}(A)$;

A gramática é LL(1), se e somente se, para cada $A \in N_G$ e $a \in \Sigma_G$, $M[A; a]$ tem no máximo uma regra.

Análise Descendente LL(1)

Usa-se uma pilha e a tabela $LL(1)$ para executar o algoritmo de parsing. O algoritmo consulta a tabela, assim ele é o mesmo para qualquer gramática $LL(1)$, o que muda é somente a tabela. O nome $LL(1)$ indica:

- A leitura é feita da esquerda para a direita (L=left-to-right);
- Procura-se construir a derivação mais a esquerda (L=leftmost);
- Exatamente 1 símbolo do resto da entrada é examinado;

Exemplo

1. $E \rightarrow T E'$
2. $T \rightarrow F T'$
3. $F \rightarrow (E)$
4. $F \rightarrow a$
5. $E' \rightarrow + T E'$
6. $E' \rightarrow \epsilon$
7. $T' \rightarrow * F T'$
8. $T' \rightarrow \epsilon$

$\text{First}(E) = \text{First}(T) = \text{First}(F) = \{ (, a \}$.

$\text{First}(E') = \{ + \}$. $\text{First}(T') = \{ * \}$.

	(	a	+	*	)	\$
E	1	1	-	-	-	-
T	2	2	-	-	-	-
F	3	4	-	-	-	-
E'	-	-	5	-	6	6
T'	-	-	8	7	8	8

$\text{Follow}(E) = \{ \$,) \}$.

$\text{Follow}(T) = \{ \$,), + \}$

$\text{Follow}(F) = \{ \$,), +, * \}$

$\text{Follow}(E') = \{ \$,) \}$

$\text{Follow}(T') = \{ \$,), + \}$

Pilha	Entrada	escolha da regra
E	a+a*a	$M[E, a] = 1$
TE'	a+a*a	$M[T, a] = 2$
FT'E'	a+a*a	$M[F, a] = 4$
aT'E'	a+a*a	-
T'E'	+a*a	$M[T', +] = 8$
E'	+a*a	$M[E', +] = 5$
+TE'	+a*a	-
TE'	a*a	$M[T, a] = 2$
FT'E'	a*a	$M[F, a] = 4$
aT'E'	a*a	-
T'E'	*a	$M[T', *] = 7$
*FT'E'	*a	-
FT'E'	a	$M[F, a] = 4$
aT'E'	a	-
T'E'	ϵ	$M[T', \$] = 8$
E'	ϵ	$M[E', \$] = 6$

Manipulação em gramáticas

A gramática do exemplo anterior foi obtida por:

- Retirada de recursão a esquerda;
- Fatoração;

As técnicas acima as vezes permitem transformar uma gramática geral livre de contexto em uma gramática LL(1), mas nem sempre.

Devemos observar que algumas gramáticas livres de contexto não têm gramáticas equivalentes LL(1), e nesse caso a aplicação dessas (ou de outras) técnicas não poderá ter sucesso.

Manipulação em gramáticas

Retirada de recursão a esquerda (relembrando...)

A eliminação da recursão à esquerda pode ser uma tentativa para transformar uma gramática em uma gramática LL(1). Observe que a combinação:

$$A \rightarrow A\alpha$$

$$A \rightarrow \beta$$

permite a geração de cadeias da forma $b\alpha\alpha\dots\alpha$, e que essas mesmas cadeias podem ser geradas de outra maneira mais aderente a LL(1).

Por exemplo,

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A'$$

$$A' \rightarrow \epsilon$$

Veja que esta outra forma de geração usa recursão à direita, que não cria os problemas que recursão à esquerda cria.

Manipulação em gramáticas

Possibilidade de fatoração (relembrando...)

Observe que este conjunto de regras:

$$A \rightarrow \alpha B$$

$$A \rightarrow \alpha C$$

Em uma gramática LL(1), dado um não-terminal e um terminal precisamos sempre ter sempre uma regra para ser aplicada. No caso acima não, quando estamos no não terminal A e lendo o terminal α , não sabemos qual regra utilizar. Nesta situação podemos fatorar para tentar transformar a gramática em LL(1):

$$A \rightarrow \alpha A'$$

$$A' \rightarrow B|C$$

Isto implica em adiar a decisão entre B e C para quando o primeiro símbolo derivado de B ou de C estiver visível.

Resumo do método/gramáticas LL(1)

Passos para construir uma tabela LL(1) e verificar se uma GLC é LL(1):

- Construir o conjunto de não-terminais que derivam ϵ ;
- Construir $First(A)$, para cada não-terminal A ;
- Construir $Follow(A)$, para cada não-terminal A ;
- Definir $M[A, a]$, a tabela LL(1), para cada não-terminal A e terminal a ;

Se, para todo A e a , $M[A, a]$ tem no máximo uma regra, então a gramática é LL(1), senão não é LL(1).

Exemplos e Exercícios (1)

```
L → L ; S | S
S → if E th L el L fi
    | if E th L fi | s
E → e
```

Exemplos e Exercícios (2)

Gramática do slide anterior fatorada e sem recursão a esquerda.

$$\begin{aligned} L &\rightarrow SL' \\ L' &\rightarrow ; S L' \mid \epsilon \\ S &\rightarrow \text{if } E \text{ th } L S' \mid s \\ S' &\rightarrow \text{el } L \text{ fi} \mid \text{fi} \\ E &\rightarrow e \end{aligned}$$

Exercício: Construir a tabela LL1 e verificar se a gramática acima é LL1.

Exemplos e Exercícios (3)

A gramática abaixo é LL1 ?

$$\begin{aligned} S &\rightarrow \text{if } E \text{ th } S \text{ el } S \mid \text{if } E \text{ th } S \mid s \\ E &\rightarrow e \end{aligned}$$

E depois da fatoração ?

1. $S \rightarrow \text{if } E \text{ th } S S'$
2. $\quad \mid s$
3. $S' \rightarrow \text{el } S$
4. $\quad \mid \varepsilon$
5. $E \rightarrow e$

Exemplos e Exercícios (4)

Após construir a tabela LL(1), verifica-se que a gramática não é LL(1)

1. $S \rightarrow \text{if } E \text{ th } S \ S'$
2. $\quad \quad | \quad s$
3. $S' \rightarrow \text{el } S$
4. $\quad \quad | \quad \varepsilon$
5. $E \rightarrow e$

	if	s	el	\$	e
S	1	2	-	-	-
S'			3/4	4	-
E					5

Observação muito importante

Uma gramática é LL(1) se passa no teste da construção da tabela LL(1)

Uma linguagem é LL(1) se existe uma gramática LL(1) que a gera.